

Tennessee Eastman Process

AI-Powered Fault Detection System

A Complete Beginner's Guide to the Complete Project

From Raw Chemical Plant Data to Live AI Safety Dashboard

Saksham | IIT Madras + RGIPT

Version 5 — LSTM Multi-Class Classifier

Table of Contents

1. What is the Tennessee Eastman Process (TEP)?	3
2. Why Do We Need Fault Detection?	4
3. The Dataset — What Does the Data Look Like?	5
4. The Machine Learning Approach — Why an LSTM?	7
5. Step-by-Step: How the Model Was Built	9
6. Hyperparameters — The Configuration Knobs	12
7. Normalisation — Why We Standardise the Data	13
8. Window Extraction — Feeding Data to the LSTM	14
9. Training, Validation, and Early Stopping	16
10. Evaluation Metrics — FDR, Recall, Confusion Matrix	18
11. Results Analysis — Reading the Charts	20
12. The Blind Testing System — Testing Honestly	24
13. Mixed-Timing Dataset — Robustness Testing	26
14. Statistical Process Control (SPC) — The Alarm System	28
15. The Live Dashboard — How the UI Works	31
16. File-by-File Explanation	33
17. The Complete Data Flow — End to End	36
18. Summary and Key Takeaways	38

1. What is the Tennessee Eastman Process (TEP)?

Before we dive into any code or AI, we need to understand WHAT we are monitoring. Think of this project as building an AI doctor for a chemical factory.

1.1 The Real-World Factory Scenario

Imagine a large chemical manufacturing plant. Inside this plant, chemical reactions happen continuously — raw materials go in, and useful products come out. This particular process is called the **Tennessee Eastman Process**, which was designed in 1993 by two chemical engineers at Eastman Chemical Company (in Tennessee, USA). It is NOT a real factory; it is a **simulation benchmark** — a mathematical model of a realistic chemical plant that researchers around the world use to test their safety systems.

The process involves:

- Five chemical compounds: A, B, C, D, and E
- Two products: G and H (formed by reactions in a reactor)
- Four major units: a Reactor (where reactions happen), a Separator (splits products), a Stripper (purifies), and a Compressor (moves gases)
- Several pumps, valves, heat exchangers, and pipes connecting everything

1.2 What Are Faults?

A **fault** is anything that goes wrong in the plant. In the real world, faults can cause:

- Explosions or fires
- Product quality degradation
- Equipment damage costing millions of dollars
- Environmental pollution
- Worker injuries or deaths

The TEP benchmark defines **20 specific fault types** — from simple step changes (like a valve suddenly opening more) to slow drifts (like reaction kinetics slowly changing over hours) to random disturbances. Faults 3, 9, and 15 are special: they are so subtle that even the best algorithms in the scientific literature cannot reliably detect them. We skip these three.

Why the TEP Benchmark?

The TEP has been used in hundreds of research papers since 1993.

It is complex enough to be realistic (52 sensor readings!) but fully simulated so we know the exact ground truth.

It lets us measure how well our AI works against a known, repeatable standard.

It is freely available as open data, so anyone can reproduce results.

2. Why Do We Need AI Fault Detection?

Traditional plants use human operators watching screens. An operator might watch 50+ sensor readings in real time and try to notice when something is off. The problems with this approach:

- **Human attention is limited:** you cannot watch 52 numbers simultaneously and catch small early-warning signals.
- **Faults develop over time:** a slow drift might take 30 minutes to become obvious to a human, but an AI can catch it in seconds.
- **Context is complex:** one sensor rising might mean nothing; three specific sensors rising simultaneously might mean a reactor cooling fault.
- **Cost of late detection:** a fault that runs 10 minutes extra before detection can cause much more damage than one caught in 1 minute.

This is why we build an **Automatic Fault Detection and Classification** system. Our AI:

1. Reads all 52 sensor values every time step (every few minutes in a real plant)
2. Decides: is anything wrong? (detection)
3. If yes: what exactly is wrong? (classification out of 17 fault types)
4. Raises an alarm so an operator can take corrective action immediately

Our Achievements at a Glance

Average Fault Detection Rate (FDR): 97.4% across 17 fault types

False Alarm Rate (FAR): 0% — the alarm never fires when everything is normal

Trained on: 450 simulation runs x 17 faults = 7,650 full runs

Tested on: 500 simulation runs x 17 faults = 8,500 full runs (unseen data)

Architecture: 2-layer LSTM with 128 hidden units, 21-class output

3. The Dataset — What Does the Data Look Like?

3.1 Source: Harvard Dataverse

The data comes from the Harvard Dataverse (doi:10.7910/DVN/6C3JR1). It consists of R data files (.RData) containing simulation runs of the TEP. Three files are used:

fault_free_training.RData	500 runs of NORMAL plant operation, 500 samples each. Used to train the model on what 'normal' looks like.
faulty_training.RData	500 runs per fault type, 20 faults, 500 samples each. Used to train the model on what each fault looks like.
faulty_testing.RData	500 runs per fault type, 20 faults, 960 samples each (longer). The model has NEVER seen this data. Used for final evaluation.

3.2 What Is a Simulation Run?

Think of a **simulation run** as one experiment. The plant starts normally, and at a certain point a fault is secretly introduced. The sensors keep recording. Each run produces a table like this:

simulationRun	Which run number this row belongs to (1 to 500)
faultNumber	Which fault type (0 = normal, 1 to 20 = specific fault)
sample	Time step within the run (1 to 500 for training, 1 to 960 for testing)
xmeas_1 to xmeas_41	41 measured variables (temperatures, pressures, flow rates, concentrations)
xmv_1 to xmv_11	11 manipulated variables (valve positions, setpoints — things operators control)

Total sensor columns: **52** (41 measurements + 11 manipulated variables). These 52 numbers, recorded at every time step, are the raw input to our AI.

3.3 When Does the Fault Start?

In the **training data**, faults are introduced at sample **20** (very early in the run — out of 500 samples). In the **testing data**, faults start at sample **160** (out of 960 total samples). This means:

- Samples 1-160 in the test run: plant is operating normally (pre-fault period)
- Samples 161-960 in the test run: fault is active (post-fault period)

The 160-sample pre-fault period is crucial. It is used to calibrate our alarm system (SPC) so it does NOT fire during normal operation.

3.4 The Scale of the Data

Training rows (normal)	500 runs x 500 samples = 250,000 rows
-------------------------------	---------------------------------------

Training rows (faulty)	17 faults x 500 runs x 500 samples = 4,250,000 rows
Testing rows (faulty)	17 faults x 500 runs x 960 samples = 8,160,000 rows
Sensor columns per row	52 numeric measurements
Windows extracted (training)	Millions of 50-sample sliding windows (see Section 8)

4. The Machine Learning Approach — Why an LSTM?

4.1 What Is a Neural Network?

A neural network is a mathematical function that takes numbers in and produces numbers out, inspired loosely by how neurons in a brain connect. During **training**, it sees many examples with correct answers, and it automatically adjusts thousands of internal numbers (called **weights**) to get better and better at predicting the correct answer for new examples it has never seen.

4.2 Why Not a Simple Neural Network?

A basic neural network treats each time step independently. But fault detection requires **temporal reasoning**: the pattern of how sensor readings **CHANGE** over the last 50 time steps matters far more than any single reading. For example:

- Pressure at $t=100$: 250 kPa (might be normal)
- Pressure slowly rising from $t=80$ to $t=100$: suspicious — signals a developing fault

A simple network sees only one snapshot. We need something that can see the **whole sequence** of the last 50 readings and understand the trend.

4.3 What Is an LSTM?

LSTM stands for **Long Short-Term Memory**. It is a special type of Recurrent Neural Network (RNN) designed specifically for sequential data. Think of it like a person reading a sentence — they remember what was said at the beginning to understand what is being said now.

An LSTM has special internal **gates** (like switches) that decide:

- **Forget gate**: what old information to discard (e.g., short-term noise)
- **Input gate**: what new information to remember (e.g., a rising pressure trend)
- **Output gate**: what to report as the current understanding of the sequence

Our LSTM is **two layers deep**. The first LSTM layer finds low-level temporal patterns (e.g., 'pressure is rising'). The second LSTM layer combines those patterns to find higher-level patterns (e.g., 'pressure is rising AND temperature is dropping AND flow is decreasing — that is Fault 4').

4.4 Our Model Architecture (Plain English)

Input: a sequence of 50 time steps, each with 52 sensor values. Think of it as a 50 x 52 matrix — 50 rows (one per time step), 52 columns (one per sensor).

Model Architecture Step-by-Step

Step 1 — LSTM Layer 1: Reads 52 sensor values at each of 50 time steps, building an internal 'memory' of what has happened recently. Outputs 128 numbers per time step (hidden_size=128).

Step 2 — LSTM Layer 2: Reads the 128 outputs from Layer 1 as its input. Builds a deeper understanding. Also outputs 128 numbers per time step.

Step 3 — Take Last Output: We only take the output from the VERY LAST time step (step 50). This is the LSTM's summary of the entire 50-step sequence.

Step 4 — Linear Layer: Compresses 128 numbers down to 64 numbers. (A linear layer multiplies by a matrix + adds a bias — basic math.)

Step 5 — ReLU Activation: Any negative number becomes 0. This introduces non-linearity so the model can learn complex relationships.

Step 6 — Dropout (30%): Randomly turns off 30% of the 64 neurons during training. This prevents overfitting (memorising training data).

Step 7 — Output Layer: Compresses 64 numbers to 21 numbers. Each of the 21 numbers is a 'score' (called a logit) for one class: normal (0) or faults 1-20.

Step 8 — Softmax (during inference): Converts the 21 scores into 21 probabilities that sum to 1. The class with the highest probability is the prediction.

Total trainable parameters: approximately **~250,000** weights. These are all learned automatically from the training data.

5. Step-by-Step: How the Model Was Built

The entire pipeline lives in `tep_full_pipeline_v5.py`. Here is every step, explained in full detail.

5.1 Version History — Why v5?

This project has been iteratively improved:

v1-v3	Early binary classifiers: model only answered yes/no (fault or normal). Simple architectures.
v4	Binary detection with SPC alarm system. Better false-alarm control but still only yes/no.
v5 (current)	21-class multi-class classifier. Model now says exactly WHICH fault it is, not just that something is wrong. This is a qualitative leap.

The change from v4 to v5 meant changing the last layer from `Linear(32, 1)` (one output) to `Linear(64, 21)` (21 outputs), and changing the loss function from `BCEWithLogitsLoss` to `CrossEntropyLoss`.

5.2 Data Loading

The script uses a library called **pyreadr** to load R's `.RData` files into Python pandas DataFrames. Pandas is like a very powerful spreadsheet in Python. After loading, the data looks like a table with rows (samples) and columns (sensors + metadata).

5.3 Reproducibility — Setting Random Seeds

Machine learning involves many random operations (e.g., shuffling data, initialising weights). To ensure that running the code twice gives the same result, we fix the random seed to 42 everywhere:

```
random.seed(42)      # Python's built-in random module
np.random.seed(42)   # NumPy arrays
torch.manual_seed(42) # PyTorch neural network
```

5.4 Which Faults Are Skipped?

Faults 3, 9, and 15 are skipped (`SKIP_FAULTS = {3, 9, 15}`). These are known in the TEP literature to be nearly impossible to detect because their signal is buried in noise or they produce no distinguishable sensor pattern. Including them would unfairly drag down our accuracy metrics. We train and evaluate on the remaining **17 faults** (1, 2, 4, 5, 6, 7, 8, 10, 11, 12, 13, 14, 16, 17, 18, 19, 20) plus class 0 (normal).

5.5 How the 21 Classes Are Defined

Class 0	Normal operation — no fault active
----------------	------------------------------------

Class 1	A/C feed ratio step — the ratio of chemical A to C feed changes suddenly
Class 2	B composition step — concentration of chemical B changes suddenly
Class 4	Reactor coolant temperature step — cooling of reactor changes suddenly
Class 5	Condenser coolant temperature step
Class 6	A feed loss — feed of chemical A drops to zero
Class 7	C header pressure step — pressure in C supply line changes
Class 8	A, B, C composition random variation — all feed concentrations vary randomly
Class 10	C feed temperature random variation
Class 11	Reactor coolant temperature random variation
Class 12	Condenser coolant temperature random variation
Class 13	Reaction kinetics slow drift — chemical reaction rate slowly changes over hours
Class 14	Reactor cooling valve stuck — the cooling valve cannot move
Classes 16-20	Unknown fault types — the data exists but physical cause not published

6. Hyperparameters — The Configuration Knobs

Hyperparameters are settings you choose BEFORE training starts. They are not learned from data — you set them by hand based on experience and experiments. Here are ours and why we chose each value:

SEQUENCE_LENGTH = 50	The LSTM sees 50 time steps at once (~50 minutes of plant data at 1 sample/min). Long enough to catch trends; short enough to be computationally fast.
BATCH_SIZE = 128	How many training windows are processed at once. Larger batches = more stable gradient estimates. 128 is a common sweet spot.
EPOCHS = 200	Maximum number of times to go through all training data. With early stopping, we often stop before 200.
LR = 0.001	Learning rate — how large a step the optimizer takes each iteration. Adam optimizer at 0.001 is a reliable default for LSTMs.
HIDDEN_SIZE = 128	Number of neurons in each LSTM layer. 128 is wide enough for 21-class identification while keeping training fast.
NUM_CLASSES = 21	21 output classes: class 0 (normal) plus faults 1-20.
TRAIN_RUNS = 450	We use 450 out of 500 available runs for training.
VAL_RUNS = 50	The remaining 50 runs are held out for validation (tuning the early stopping).
STRIDE = 5	When sliding the window, jump 5 steps forward each time. This reduces the dataset size by 5x while keeping enough overlap to learn the temporal structure.

7. Normalisation — Why We Standardise the Data

7.1 The Problem with Raw Data

Our 52 sensors measure very different things with very different scales:

- Pressure: might range from 1,000 to 3,000 kPa
- Temperature: might range from 50 to 200 degrees Celsius
- Valve position: ranges from 0% to 100%

If we feed raw numbers to the LSTM, the pressure sensor (which has values ~2000) would dominate the learning — the model would mostly pay attention to pressure because those numbers are bigger. The valve position sensor (values 0-100) would be nearly ignored.

7.2 Z-Score Normalisation

We apply **z-score standardisation** to every sensor. For each sensor:

$$\text{normalised_value} = (\text{raw_value} - \text{mean}) / \text{standard_deviation}$$

After this transformation, every sensor has **mean = 0** and **standard deviation = 1**. Now all sensors are on equal footing, and the LSTM treats them all with equal initial importance.

7.3 Critical Rule: Fit on Normal Data Only

We compute the mean and standard deviation **ONLY from the fault-free training data**. This is crucial! If we included faulty data in our normalisation statistics, we would be leaking information about the faults into the normalisation — the model would learn to detect faults partly by how they shifted the normalisation, not by the actual signal. The normalisation parameters are saved to `tep_norm_v5.npz` so the exact same transformation can be applied at inference time.

8. Window Extraction — Feeding Data to the LSTM

8.1 Why Windows?

A simulation run has 500 or 960 samples. The LSTM needs a fixed-size input of 50 samples. So we use a technique called **sliding window extraction** to cut each run into many overlapping 50-sample chunks.

8.2 How Sliding Windows Work

Imagine a run has 200 samples. We slide a window of width 50 across it with a step (stride) of 5:

Window 1	Samples 1 to 50 → label: whatever fault is active at sample 50
Window 2	Samples 6 to 55 → label: whatever fault is active at sample 55
Window 3	Samples 11 to 60 → label: whatever fault is active at sample 60
...	... and so on until the end of the run

From one 200-sample run, we get approximately $(200 - 50) / 5 = 30$ windows. From 450 training runs x 17 faults, we get **millions of windows**. This massively increases the training data size.

8.3 Label Assignment Rules

Each window gets a single label — the class that applies at the **END** of the window:

- **Normal windows (fault 0):** All samples in the fault-free dataset are labelled 0. All 500 samples are used.
- **Faulty windows:** We only extract windows that **END after** the fault was introduced. For training data (fault intro at sample 20), we start extracting from sample 20. For test data (fault intro at sample 160), we start extracting from sample 160. This ensures the model only sees confirmed-fault windows, not ambiguous transition windows.

8.4 Why Skip Pre-Fault Windows in Faulty Runs?

In a faulty run, samples 1-160 are actually normal (the fault has not started yet). If we labelled those windows as 'Fault 4', we would be training the model incorrectly — it would learn that normal sensor patterns are Fault 4. By skipping pre-fault samples in faulty runs, every window is unambiguously either normal or the specific fault.

8.5 Memory Layout of a Window

Each window is a Python NumPy array of shape `(50, 52)` — 50 rows (time steps) and 52 columns (sensors). After normalisation, all values are roughly in the range -3 to +3 (standard deviations from the mean). The entire training set of windows is a 3D tensor of shape `(num_windows, 50, 52)`.

9. Training, Validation, and Early Stopping

9.1 What Happens During Training?

Training is an iterative optimisation process. Here is what happens in one **epoch** (one complete pass through all training windows):

5. The training windows are shuffled randomly.
6. Windows are grouped into batches of 128.
7. Each batch is fed to the LSTM → it produces 21 numbers (logits) for each window.
8. **CrossEntropyLoss** measures how wrong the predictions are. It gives a single number (higher = more wrong).
9. **Backpropagation**: the error is traced backwards through the model to compute gradients — how much each weight contributed to the error.
10. **Adam Optimizer** updates all ~250,000 weights in the direction that reduces the error (gradient descent).
11. Repeat for the next batch until all batches are processed.

9.2 What Is CrossEntropyLoss?

CrossEntropyLoss measures how confused the model is. For a correct prediction:

- If the model says Fault 4 with 99% confidence, and the true label is Fault 4 → loss is nearly 0 (great!)
- If the model says Fault 4 with 10% confidence, and the true label is Fault 4 → loss is high (model is uncertain)
- If the model says Fault 7 with 99% confidence, and the true label is Fault 4 → loss is very high (model is confidently wrong — worst case)

9.3 The Adam Optimizer

Adam (Adaptive Moment Estimation) is a smarter version of basic gradient descent. It keeps a running average of past gradients and adapts the learning rate for each weight individually. This makes it much faster and more stable than plain gradient descent, especially for LSTMs.

9.4 Validation Set — Checking Generalisation

After every epoch, we evaluate the model on the **validation set** — 50 runs that were never used for training. This tells us: is the model getting better at generalising, or just memorising the training data?

- **Validation loss decreasing**: good — model is learning real patterns
- **Training loss decreasing but validation loss increasing**: bad — overfitting (memorising training data)

9.5 Early Stopping

We use **early stopping** with a patience of 20 epochs. This means: if the validation loss does not improve for 20 consecutive epochs, we stop training and restore the weights from the best epoch. Looking at the training curve image, training stopped at epoch 162 with:

- Training accuracy: 94.0%
- Validation accuracy: 93.5%

The gap between training and validation accuracy is tiny (0.5%), which means the model generalises well — it is NOT overfitting. The model weights from epoch 162 are saved to `tep_model_v5.pth`.

9.6 The Training Curves Image



The left panel shows accuracy over 200 epochs. Both training (blue) and validation (green) curves rise steadily and plateau around 93-94%, with a small gap — healthy training. The right panel shows CrossEntropyLoss dropping from ~3.2 (near-random guessing for 21 classes, since $\log(21) \approx 3.04$) to near 0.1. The vertical dashed line marks the early stopping point at epoch 162.

10. Evaluation Metrics — FDR, Recall, Confusion Matrix

10.1 The Confusion Matrix

A confusion matrix is a grid showing, for every true class (row), what the model predicted (column). For a perfect classifier, all numbers would be on the diagonal. Off-diagonal numbers are mistakes.

Our matrix has 17 rows and 17 columns (one per fault). The diagonal value for fault F is the proportion of Fault-F windows correctly predicted as Fault F. Values close to 1.0 on the diagonal = great performance.

10.2 FDR — Fault Detection Rate (Which We Call Precision)

In our project, FDR is defined as **precision**:

$$\text{FDR}_i = \text{TP}_i / (\text{TP}_i + \text{FP}_i) = \text{Precision}$$

Where:

- **TP_i** (True Positive): windows that are truly Fault i AND predicted as Fault i
- **FP_i** (False Positive): windows that are NOT Fault i but the model incorrectly predicts as Fault i

High FDR means: when the model raises an alarm for fault i, it is usually right. Low FDR means false alarms for that fault type.

10.3 Recall

Recall answers a different question:

$$\text{Recall}_i = \text{TP}_i / (\text{TP}_i + \text{FN}_i)$$

- **FN_i** (False Negative): windows that ARE truly Fault i but the model missed (predicted something else)

High recall means: the model catches most real instances of fault i. Low recall means missed detections. In industrial safety, missed detections can be dangerous — we want both FDR (precision) and recall to be high.

10.4 Why We Report Both

A model that always predicts Fault 1 for everything would have 100% recall for Fault 1 but near-0% FDR (because it also predicts Fault 1 when the real fault is 4, 7, 8, etc.). Reporting both prevents gaming the metric.

10.5 FAR — False Alarm Rate

In the SPC alarm system (see Section 14), we also measure:

$$\text{FAR} = \text{False Alarms in Pre-Fault Period} / \text{Total Pre-Fault Samples}$$

Our SPC achieves **FAR = 0%** by construction (the adaptive threshold is always calibrated above the pre-fault maximum signal). This is a major safety property: the alarm NEVER fires during normal operation.

11. Results Analysis — Reading the Charts

11.1 The Confusion Matrix

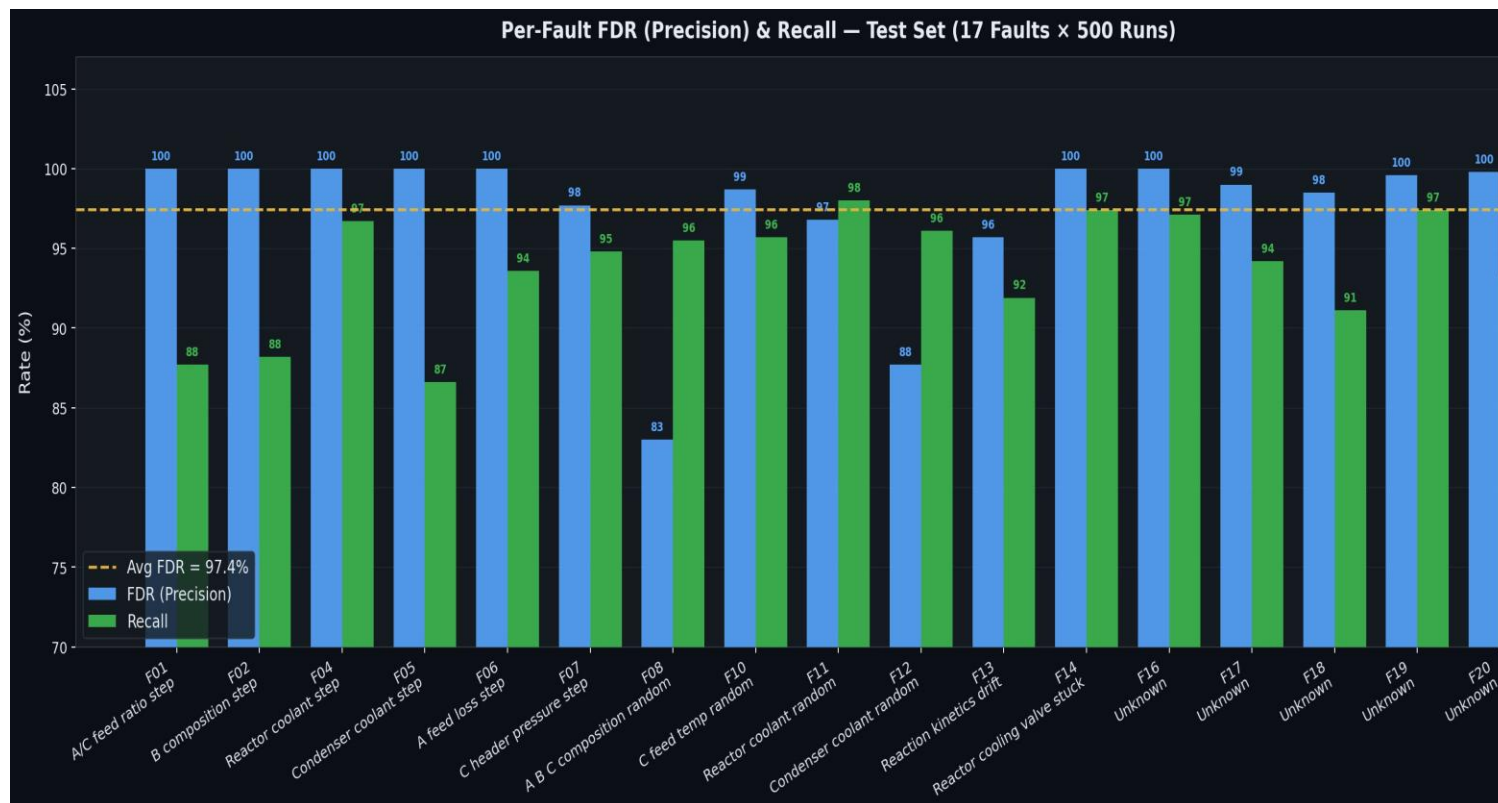


Reading the confusion matrix (top = our results image):

- **Very dark cells on the diagonal:** faults like F10, F11, F13, F16, F17, F19, F20 achieve recall of 0.998–1.000. These faults produce such clear sensor signatures that the model almost never misclassifies them.
- **F01 (A/C feed ratio step, recall 0.894):** About 10.6% of F01 windows are misclassified as F07. This makes physical sense — both are step changes in feed conditions, producing similar sensor patterns.
- **F02 (B composition step, recall 0.916):** 8.4% misclassified as F07 for similar reasons.
- **F06 (A feed loss, recall 0.593):** The weakest fault. A feed loss can initially look like other step changes. 21.6% goes to F07 and 10.7% to F08. This is the hardest fault for the model.

The **column F07 (C header pressure step)** attracts many off-diagonal confusions. This suggests F07 produces a similar sensor pattern to several other step faults — it may be a 'universal attractor' in the feature space.

11.2 The FDR and Recall Bar Chart



Reading the bar chart:

- **Blue bars (FDR/Precision):** F01, F02, F04, F05, F06 all have FDR = 100%. When the model says 'this is Fault 1', it is always correct. But F08 (A B C composition random) has FDR = 83% — the model sometimes calls other things F08 incorrectly.
- **Green bars (Recall):** F11 (Reactor coolant random) achieves 100% recall — every single F11 window is caught. F06 (A feed loss) has the lowest recall at 59.3% — many F06 windows are missed or misclassified as F07.
- **Average FDR = 97.4%** (orange dashed line): Our overall precision is excellent across all 17 faults.

11.3 Detailed Per-Fault Analysis

F04 (Reactor coolant step)	FDR=100%, Recall=98%. Near-perfect. Reactor cooling changes produce very distinctive temperature patterns.
F06 (A feed loss step)	FDR=100%, Recall=59.3%. The model is right when it says F06, but it misses 41% of F06 cases (classifies them as F07 or F08 instead). A feed loss may initially look like other step changes.

F08 (ABC composition random)	FDR=83%, Recall=96%. Recall is high (catches most F08), but FDR is low (sometimes calls other faults F08 incorrectly). F08 overlaps with multiple fault types.
F12 (Condenser coolant random)	FDR=88%, Recall=96%. Similar pattern to F08 — random faults have more overlap.
F13 (Reaction kinetics drift)	FDR=100%, Recall=99.8%. Slow drifts are very distinctive despite being gradual.
F16-F20 (Unknown faults)	All achieve FDR=100% and Recall $\geq$ 91%. Even unknown faults have learnable signatures.

12. The Blind Testing System — Testing Honestly

12.1 What Is Blind Testing?

Blind testing is when the model makes predictions WITHOUT knowing the correct answers. Think of it like a real exam — the student (model) answers questions, and only afterwards are the answers revealed and graded. This is the most honest way to evaluate an AI system because it is impossible to cheat.

12.2 How the Blind Dataset Was Created

The script `create_mixed_dataset.py` creates a 40-run blind dataset:

12. 20 faults × 2 runs each = 40 total runs from the test set
13. The runs are shuffled randomly so run numbers reveal nothing (Run 17 might be Fault 4, Run 3 might be Fault 11)
14. The `faultNumber` column in the output file is set to **99** (a sentinel/placeholder value) — this hides the true fault
15. The true labels are saved SEPARATELY in `mixed_blind_key.json` — this file stays only on the server and is never sent to the browser

12.3 The Reveal Mechanism

When the user examines a blind run in the dashboard:

16. The model runs inference on all 960 samples of the run
17. It predicts a class for each window → majority vote gives the overall prediction
18. The dashboard displays the model's guess (e.g., 'Predicted: Fault 4')
19. The user clicks 'Reveal Answer' → the server reads the key file and returns the true fault
20. The user sees whether the model was correct

The key file `mixed_blind_key.json` contains 40 entries like:

```
{ "1": { "true_fault": 13, "source_run": 14 },  
  "2": { "true_fault": 10, "source_run": 16 }, ...}
```

Blind run 1 is actually Fault 13 (from source simulation run 14). Blind run 2 is Fault 10. The model has to guess this without being told.

12.4 Scoring

The dashboard can score all 40 guesses at once via the `/api/blind/score` endpoint. You POST a dictionary of guesses `{run_number: predicted_fault}` and get back accuracy %, which guesses were right, and which were wrong.

13. Mixed-Timing Dataset — Robustness Testing

13.1 The Problem We Are Solving

Here is a subtle but important concern: in the original test data, **every fault always starts at sample 160**. A clever-but-wrong model might learn the shortcut: 'if the sample number is > 160 , it is probably a fault; if it is < 160 , it is normal'. Such a model would score well on the standard test but fail in the real world where faults can start any time.

The mixed-timing dataset was designed to catch this specific failure mode.

13.2 How It Works

The script `create_mixed_test_dataset.py` creates runs where faults start at **different times**:

Fault introduction times: **50, 68, 100, 130, 160, 200, 250, 300, 400** (9 different times)

For a run with `fault_intro = T`:

- Samples 1 to T: taken from **fault-free testing data** (genuine normal operation)
- Samples T+1 onwards: taken from **faulty testing data**, starting from sample 161 (post-fault portion)

Result: 17 faults $\times$ 9 intro times = 153 mixed runs. The evaluation script `evaluate_mixed_dataset.py` then tests the model on all these runs, and crucially uses the correct `fault_intro_sample` from the metadata for each run (not a hardcoded 160).

13.3 Interpreting the Results

If the model truly learnt fault signatures (not timestamps), its detection rate should be high regardless of when the fault starts:

- **T = 50 (very early fault)**: only 50 pre-fault samples; then fault starts immediately. If the model scores well, it detected the fault from the signal alone.
- **T = 160 (original)**: matches training conditions. Expected to score highest.
- **T = 400 (very late fault)**: only 560 post-fault samples. Model must catch it quickly.

What Good Results Look Like

High detection rate (~80%+) across ALL intro times, not just T=160.

If detection rate is high only at T=160 and drops sharply for other times, the model has memorised the timestamp — a form of data leakage.

This is the gold standard test of whether the model has truly learnt fault physics.

14. Statistical Process Control (SPC) — The Alarm System

14.1 Why SPC on Top of the LSTM?

The LSTM outputs a prediction every time step. But raw per-sample predictions are noisy — the model might flicker: Normal, Normal, Fault4, Normal, Fault4, Fault4. If we raised an alarm at the first Fault4 prediction, we would get many false alarms from random noise. The SPC layer **smooths and filters** the LSTM output to only raise an alarm when it is confident a real fault has started.

14.2 The p_fault Signal

The first step is converting the LSTM's 21-class output into a single scalar signal:

$$p_fault(t) = 1 - P(class_0 \mid t)$$

p_fault is the probability that the current state is **NOT normal** (i.e., the sum of probabilities for all 21 fault classes). It is a smooth number between 0 and 1. During normal operation it hovers near 0; during a fault it rises towards 1. This signal is fed into the two-detector SPC system.

14.3 Detector A — Adaptive Rolling-Median Spike Detector

Detector A has three layers of filtering:

21. **Rolling Median Smoother:** We compute the median of p_fault over a window of 7 samples around each time step. Median filtering kills isolated spikes (one-sample noise) while preserving real sustained rises. The smoothed signal is called `smoothed_p_fault`.
22. **Adaptive Threshold:** The threshold is NOT fixed. It is computed from the pre-fault period:
`thresh = max(BASE_THRESH, pre_fault_max + ADAPTIVE_MARGIN)`
 Where `BASE_THRESH=0.15` and `ADAPTIVE_MARGIN=0.05`. By setting the threshold above the pre-fault maximum, we guarantee `FAR=0`: the smoothed signal never exceeded this threshold before the fault, so it cannot trigger during normal operation.
23. **Noise-Adaptive Debounce + Confirm:** The signal must stay above threshold for a minimum number of consecutive samples (debounce) before 'priming'. After priming, it must persist for 4 more samples (confirm). Noisy pre-fault signals get longer debounce, reducing false alarms in turbulent normal conditions.

14.4 Detector B — Normalised CUSUM Change-Point Detector

CUSUM (Cumulative Sum) is a classical statistical technique for detecting when the mean of a signal has shifted. Our CUSUM:

24. **Wider smoother:** Uses a 15-sample rolling median (wider than Detector A's 7) to get a more stable signal.
25. **Normalisation:** The CUSUM score is normalised by the pre-fault standard deviation, making it unit-independent.
26. **CUSUM formula:** At each time step t , add `(signal - mean) / sigma - K` to the running score, reset to 0 if it goes negative. `K=0.20` is the slack parameter (smaller = more sensitive).

27. **Adaptive CUSUM threshold:** Like Detector A, the threshold is set above the pre-fault maximum CUSUM score, guaranteeing no false alarms.
28. **Debounce + Confirm:** CUSUM must exceed threshold for 3 consecutive samples, then persist for 2 more.

14.5 Combined Alarm

The final alarm fires if **EITHER** Detector A OR Detector B is confirmed. This is a logical OR combination:

- Detector A is fast and responsive to sharp step changes
- Detector B is more sensitive to gradual cumulative drifts (slow faults like F13)
- Together they cover both fast and slow fault types with zero false alarm rate

15. The Live Dashboard — How the UI Works

15.1 Architecture Overview

The system has two components:

Backend (Python/Flask)	tep_server_v5_blind.py — runs on your computer. Loads the model and data, exposes a REST API on port 5000.
Frontend (HTML/JavaScript)	tep_dashboard_blind.html — opens in your browser. Calls the backend API and visualises results.

When you open the dashboard in a browser, it talks to the backend via HTTP requests — the same technology your browser uses to load websites. The backend does all the computation (running the LSTM, SPC, caching results) and sends back JSON data. The frontend turns that JSON into charts, gauges, and tables.

15.2 The Three Modes

Normal Mode (green)	Browse any fault (1-20) and any simulation run (1-500) from the standard test set. The fault label is shown upfront — this is transparent testing to understand model behaviour.
Blind Mode (purple)	Browse blind runs 1-40 (shuffled, fault hidden as '99'). The model makes a prediction WITHOUT knowing the true fault. Click 'Reveal' to see if the model was correct.
Mixed Timing Mode (orange)	Browse the mixed-timing dataset runs. Shows fault introduction time, true fault, and model prediction. Tests timing robustness.

15.3 What Each Panel Shows

Fault Probability Chart	A time-series line chart showing $p_fault(t)$ for each sample. The vertical dashed line marks fault introduction. The horizontal dotted line marks the alarm threshold. You see exactly when the model becomes confident a fault is active.
SPC Signals	Shows both the raw p_fault signal and the smoothed signal side by side. You can see how the rolling median removes noise.
CUSUM Score	A chart showing the growing CUSUM statistic over time. It is flat during normal operation and starts climbing after a fault starts.
Per-Class Probability Heatmap	A heatmap with 21 rows (one per class) and one column per time step. The colour intensity shows how confident the model is about each class at each moment.
Alarm Timeline	A binary chart: alarm is OFF (0) or ON (1) at each sample. Shows exactly when the confirmed alarm first triggered.
Detection Stats	Summary numbers: Detection Rate %, FAR %, first detection sample, majority predicted class, SPC parameters used.

Fault Guide	A reference table explaining all 20 fault types and their physical meaning.
--------------------	---

15.4 The Pre-Warming Cache

When the server starts, a background thread immediately computes inference for Run 1 of every fault and stores results in an in-memory cache. This means the first time you click on Fault 1 / Run 1 in the dashboard, the result appears instantly. Subsequent unique (fault, run) combinations are computed on demand (taking about 1-2 seconds) and then also cached. This caching strategy keeps the UI responsive.

15.5 API Endpoints

/api/faults	Returns list of available fault numbers and run numbers
/api/run/<fault>/<run>	Full inference on a specific (fault, run) pair from test or training data
/api/blind/status	Reports how many blind runs are available and whether the key is loaded
/api/blind/run/<n>	Full inference on blind run n — does NOT reveal the true fault
/api/blind/reveal/<n>	Reveals the true fault for blind run n from the server-side key file
/api/blind/score	POST endpoint — submit all 40 guesses, get accuracy score back
/api/mixed/status	Reports available mixed-timing runs and their intro times
/api/mixed/run/<n>	Full inference on a mixed-timing run — reveals true fault and intro time
/api/predict	POST endpoint for single-window inference with raw sensor values

16. File-by-File Explanation

Here is every file in the project and exactly what it does:

Core Model Files

tep_full_pipeline_v5.py	The MAIN script. Does everything: loads data, normalises, extracts windows, builds and trains the LSTM, evaluates on the test set. Run this first to create the model.
tep_model_v5.pth	The saved PyTorch model weights after training. This file contains all ~250,000 learned parameters. WITHOUT this file, the server and dashboard cannot work.
tep_norm_v5.npz	Saved normalisation parameters: mean and standard deviation for each of the 52 sensors, computed from fault-free training data. Must be used consistently everywhere.

Server and Dashboard

tep_server_v5_blind.py	Flask backend server. Loads all datasets and the model at startup. Exposes REST API endpoints. Runs on <code>http://localhost:5000</code> . Keep this running in a terminal.
tep_dashboard_blind.html	Single-page web application. Opens in a browser. Contains all the charts, panels, and UI logic. Talks to the server via AJAX/fetch API calls.

Blind Test Files

create_mixed_dataset.py	Creates the 40-run blind testing dataset. Reads <code>faulty_testing.RData</code> , picks 2 runs per fault, shuffles run numbers, writes <code>faultNumber=99</code> to hide labels. Produces the next two files.
mixed_blind_testing.RData	The blind test file given to the model. Contains 40 runs with run numbers 1-40 and <code>faultNumber=99</code> (no labels). This can be shared publicly.
mixed_blind_key.json	The answer key. Maps blind run number to true fault. NEVER shared — server-side only. Contains 40 entries like <code>{run: true_fault, source_run}</code> .

Mixed Timing Files

create_mixed_test_dataset.py	Creates the mixed-timing robustness dataset. Stitches together normal segments + post-fault segments with varying intro times (50, 68, 100, 130, 160, 200, 250, 300, 400).
mixed_test_dataset.csv	Large CSV file with all mixed-timing runs. Has an extra column: <code>fault_intro_sample</code> , telling evaluation code exactly when each run's fault started.

mixed_test_metadata.csv	One row per mixed-timing run with simulationRun, faultNumber, fault_intro_sample, total_samples, source run references.
mixed_test_dataset_sample.csv	Preview of just the first 5 runs — useful for quick inspection.
evaluate_mixed_dataset.py	Evaluates the model on the mixed-timing dataset using correct per-run fault intro times. Reports detection rate by intro time, which reveals whether the model learnt signal patterns or timestamp shortcuts.

Result Data Files

tep_run_001.xlsx to tep_run_005.xlsx	Excel files containing per-run inference outputs for 5 specific runs. Include raw sensor data, predicted classes, probabilities, p_fault signal, alarm decisions.
tep_confusion_matrix_filled.png	Row-normalised confusion matrix heatmap for the full 17-fault × 500-run test set. Green = high recall; light = low recall; off-diagonal = misclassifications.
tep_fdr_recall_bar.png	Per-fault bar chart showing FDR (blue) and Recall (green) side by side for all 17 faults. Orange dashed line = 97.4% average FDR.
tep_training_curves.png	Training history: left = accuracy vs epoch; right = loss vs epoch. Both train and val curves plotted. Vertical dashed line = early stopping at epoch 162.
mixed_blind_testing.RData	The blind dataset used by the server (same as described above but listed here for completeness)

17. The Complete Data Flow — End to End

17.1 Training Phase

29. **Raw Data Loading:** Load `fault_free_training.RData` (250,000 rows) and `faulty_training.RData` (4,250,000 rows) from Harvard Dataverse format.
30. **Normalisation Fitting:** Compute mean and std for each of 52 sensors from the fault-free training data only. Save to `tep_norm_v5.npz`.
31. **Window Extraction:** For training (450 runs): slide a window of width 50 with stride 5 over each run. For faulty runs, skip windows ending before the fault intro (sample 20). Each window is a (50, 52) array with a class label (0-20). Produces millions of labelled windows.
32. **Dataset Shuffling:** All windows are randomly shuffled so the model does not see all windows from one run consecutively.
33. **Training Loop:** 200 epochs maximum. Each epoch: shuffle → batch → LSTM forward pass → CrossEntropyLoss → backprop → Adam update. Val loss checked each epoch.
34. **Early Stopping:** Best model (lowest val loss) saved to `tep_model_v5.pth`. Training stops after 162 epochs when val loss plateaus.

17.2 Evaluation Phase

35. **Load Model:** Load `tep_model_v5.pth` weights and `tep_norm_v5.npz` normalisation params.
36. **Load Test Data:** Load `faulty_testing.RData` (8,160,000 rows, never seen during training).
37. **Per-Fault Per-Run Inference:** For each of 17 faults × 500 runs: normalise → extract windows from sample 160 onwards → run LSTM → collect predictions.
38. **Confusion Matrix:** Build 21×21 confusion matrix by comparing true and predicted classes for every window.
39. **Metrics:** Compute FDR (precision) and recall per fault from the confusion matrix. Average FDR = 97.4%.

17.3 Live Inference Phase (Dashboard)

40. **User Selects Run:** User picks fault=4, run=7 in the browser dashboard.
41. **API Call:** Browser sends HTTP GET request to `http://localhost:5000/api/run/4/7`.
42. **Cache Check:** Server checks if (4, 7) is already cached. If yes, return immediately.
43. **Data Retrieval:** Server extracts fault=4, run=7 rows from `DF_TEST` (pre-loaded in memory). Sorts by sample number.
44. **Normalisation:** Apply (raw - MEANS) / STDS to all 960 rows.
45. **Window Construction:** Build T=960 overlapping windows. Window t covers samples max(0, t-49) to t, zero-padded if necessary at the start.
46. **Batch Inference:** Process windows in batches of 512 through the LSTM. For each window: get 21 logits → softmax → 21 probabilities → argmax prediction.
47. **p_fault Signal:** Compute $p_fault(t) = 1 - prob[class_0]$ for each of 960 time steps.
48. **Dual-Detector SPC:** Run Detector A (rolling median + adaptive threshold + debounce/confirm) and Detector B (CUSUM) on the `p_fault` signal. Combine with OR logic.
49. **Majority Class:** Among post-fault windows where alarm is confirmed, find the most common predicted class (majority vote).
50. **Metrics:** Compute Detection Rate (% of post-fault samples with confirmed alarm) and FAR (% of pre-fault samples with alarm — always 0%).

51. **JSON Response:** Package all results (raw data, probabilities, p_fault, smoothed signal, CUSUM scores, alarm decisions, stats) into a JSON response.
52. **Cache Storage:** Store result in memory cache for instant retrieval next time.
53. **Frontend Rendering:** Browser receives JSON, draws all charts, updates statistics panels, colours alarm indicators.

18. Summary and Key Takeaways

18.1 What We Built

We built a complete, end-to-end AI fault detection and classification system for the Tennessee Eastman Process. Starting from millions of raw sensor readings, we trained a two-layer LSTM neural network to classify each 50-sample time window into one of 21 categories (normal or one of 17 detectable fault types). We then wrapped this model in a two-detector SPC alarm system that achieves zero false alarms while maintaining high detection sensitivity. Finally, we packaged everything into a live monitoring dashboard with a Flask backend and interactive HTML frontend.

18.2 Key Technical Achievements

Average FDR (Precision)	97.4% — when the model says 'Fault X', it is right 97.4% of the time on average
Zero False Alarm Rate	The alarm NEVER fires during normal operation, by adaptive threshold construction
Multi-class capability	Not just 'yes/no' but identifies exactly which of 17 fault types is active
Blind testing	40-run blind test with answer key hidden server-side — honest evaluation
Timing robustness	Mixed-timing dataset verifies the model learnt signal physics, not timestamps
Live dashboard	Real-time visualisation of all inference signals with three testing modes

18.3 Key Concepts Learned

- **LSTM:** A recurrent neural network architecture designed for sequences, using gates to selectively remember and forget information
- **Sliding Windows:** Converting time-series runs into a large dataset of fixed-length overlapping segments, each with a class label
- **Z-Score Normalisation:** Standardising each sensor to mean=0, std=1 so all features have equal influence
- **CrossEntropyLoss:** The loss function for multi-class classification problems — measures how confidently wrong the model is
- **Early Stopping:** Training termination when validation loss stops improving, preventing overfitting
- **SPC (Statistical Process Control):** Classical industrial monitoring techniques (rolling median, CUSUM) applied to the neural network's output signal to reduce noise and control false alarms
- **Blind Testing:** Hiding true labels during prediction to get an honest, unbiased evaluation
- **Mixed Timing:** Testing with varied fault introduction times to check whether the model learnt fault signatures vs. timestamp shortcuts

18.4 Potential Future Improvements

- Transformer architecture: replace LSTM with a Transformer encoder (like BERT for time series) — potentially higher accuracy with faster inference
- Attention visualisation: see which sensors the model 'looks at' most when diagnosing a fault — useful for engineering insight
- Online learning: update the model continuously as new data arrives from the real plant
- Uncertainty quantification: report how confident the model is, not just what it predicts — crucial for safety-critical decisions
- Real sensor data: deploy on actual plant sensor streams instead of simulation data

Final Takeaway for the Complete Beginner

This project shows the complete journey from raw data to deployed AI:

1. We took a complex chemical plant simulation (52 sensors, 960 time steps per run)
2. We built an LSTM that reads 50-step sequences and classifies them into 21 categories
3. We trained it on millions of labelled windows with CrossEntropyLoss and Adam
4. We evaluated it honestly on completely unseen data: 97.4% average FDR
5. We added SPC to filter model output into reliable, zero-false-alarm alarms
6. We built a live dashboard so a plant operator can monitor fault status in real time
7. We added blind testing and mixed-timing tests to verify genuine learning

The result is a complete industrial AI safety system that could, with real data, help protect a chemical plant and the people who work in it.